



Lab #7 - Logic Operations

Logical Instructions:

Instruction	Description	Example	Valid Operand Combinations
AND	Performs a bitwise AND operation between two operands. Only sets bits in the result if both bits are 1.	AND EAX, EBX ; Bitwise AND between EAX and EBX.	AND reg, reg AND reg, imm AND reg, mem
OR	Performs a bitwise OR operation between two operands. Sets a bit if either or both bits are 1.	OR EAX, EBX ; Bitwise OR between EAX and EBX.	OR reg, reg OR reg, imm OR reg, mem
XOR	Performs a bitwise XOR (exclusive OR) operation. Sets a bit if one (but not both) of the bits is 1.	XOR EAX, EBX ; Bitwise XOR between EAX and EBX.	XOR reg, reg XOR reg, imm XOR reg, mem
NOT	Performs a bitwise NOT operation, flipping all bits (1s become 0s and vice versa).	NOT EAX ; Bitwise negation of EAX (inverting all bits).	NOT reg NOT mem
TEST	Performs a bitwise AND operation between two operands, but doesn't modify the destination operand. Updates the flags according to the result, but does not store the result.	TEST EAX, EBX ; Tests the bitwise AND of EAX and EBX.	TEST reg, reg TEST reg, imm TEST reg, mem

Comparison:

The CMP (compare) instruction in x86 assembly is used to compare two values by performing a subtraction between the two operands, but it does not store the result. Instead, it updates the CPU's flags based on the result of the subtraction, which can then be used to make decisions via conditional jumps (like JE, JNE, JL, JLE, JA, JB, etc.). The two operands can be registers, immediate values, or memory locations. The flags that are primarily affected by CMP include the Zero Flag (ZF), Sign Flag (SF), Overflow Flag (OF), and Carry Flag (CF).

The behavior of CMP is the same as the SUB instruction but without storing the result in a destination register. After a CMP, the subsequent instructions that follow can check the flags to determine the relationship between the two values, whether they are equal, one is greater, or one is less.

CMP op1, op2

CMP Instruction

- When two unsigned operands are compared, the Zero flag and Carry flag indicate the following relations between operands:

CMP Results	ZF	CF
Destination < source	0	1
Destination > source	0	0
Destination = source	1	0

- When two signed operands are compared, the Sign, Zero, and Overflow flags indicate the following relations between operands:

CMP Results	Flags
Destination < source	SF ≠ OF
Destination > source	SF = OF
Destination = source	ZF = 1

18

Figure 1. Course slides - Topic 5 Part 2 - Slide 18

The JMP Instruction and Conditional Jumps in Assembly:

In x86 assembly, the JMP instruction is an unconditional jump that transfers control to a specified label or address. It's useful for implementing loops, skipping sections of code, or transferring control to different parts of a program without evaluating any conditions. In addition to JMP, there are conditional jump instructions like JZ, JNZ, JE, and JNE. These conditional jump instructions allow control flow to branch based on the results of previous instructions, typically following a CMP instruction that sets the appropriate flags. For instance, JZ (Jump if Zero) will make the program jump to a specified label if the Zero Flag (ZF) is set, which indicates that a comparison resulted in equality. These instructions are essential for making decisions and implementing control flow mechanisms like loops and conditionals in assembly programs.

Innstruction	Description	Example
JMP	Unconditionally jumps to a specified label or address.	JMP label1
JNZ	Jumps if Zero Flag (ZF) is not set (result is non-zero).	JNZ label2 after comparing values.
JZ	Jumps if Zero Flag (ZF) is set (result is zero).	JZ label3 after a comparison yields equality.
JE	Jumps if Equal (equivalent to JZ).	JE label4 after CMP EAX, EBX if equal.
JNE	Jumps if Not Equal (equivalent to JNZ).	JNE label5 after CMP EAX, EBX if not equal.

Example Assembly Code with Jumps

Below is an example of how to use JMP and conditional jumps like JZ and JNZ in assembly:

```
INCLUDE Irvine32.inc

.data
    num1 DWORD 10
    num2 DWORD 20
    resultEqual BYTE "Numbers are equal", 0
    resultNotEqual BYTE "Numbers are not equal", 0
```

```

.code
main PROC
    ; Load numbers into registers
    MOV EAX, num1
    MOV EBX, num2

    ; Compare the numbers
    CMP EAX, EBX

    ; Jump if equal
    JE equal

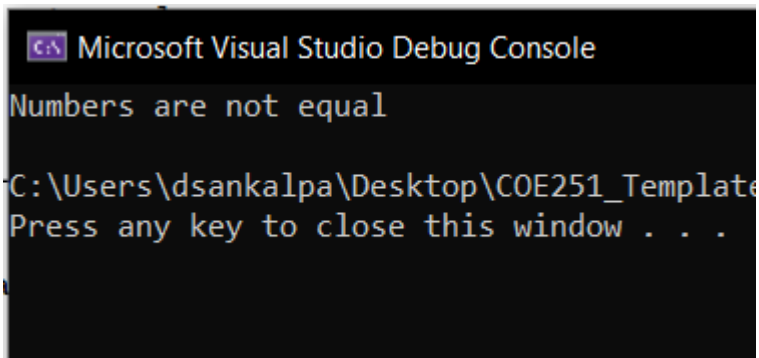
    ; Jump if not equal (this is executed if the JE condition is false)
    JNE not_equal

equal:
    MOV EDX, OFFSET resultEqual
    CALL WriteString
    JMP done

not_equal:
    MOV EDX, OFFSET resultNotEqual
    CALL WriteString

done:
    CALL CrLf
    EXIT
main ENDP
END main

```



Lab Work (Questions 1-5):

1. Write an assembly program that reads 5 integers from the user and counts how many of the numbers are even and how many are odd. Use the **TEST** instruction to check if a number is even (if the least significant bit is 0) or odd. Display the even and odd numbers as well as the final counts of even and odd numbers. Assume that the user will always input both odd and even number.

```
Microsoft Visual Studio Debug Console
Enter a number: 24
Enter a number: -26
Enter a number: 1109
Enter a number: 53
Enter a number: -5
2 even numbers: +24 -26
3 odd numbers: +1109 +53 -5

C:\Users\dsankalpa\Desktop\COE251_Template\COE251_T
Press any key to close this window . . .
```

Solution:

TITLE Name/Purpose Here

INCLUDE Irvine32.inc

.data

```
str1 byte "Enter a number: ", 0
str2 byte "even numbers: ", 0
str3 byte "odd numbers: ", 0
str4 byte " ", 0
arr dword 5 DUP(?)
even_arr dword 5 DUP(?)
odd_arr dword 5 DUP(?)
evenCount dword 0
oddCount dword 0
```

.code

main PROC

```
mov edx, offset str1
```

```
mov esi, offset arr
mov ecx, lengthof arr
```

```
first:
call writestring
call readint
mov [esi], eax
add esi, 4
loop first
```

```
mov esi, offset arr
mov ecx, lengthof arr
```

```
mov edi, offset even_arr
mov ebx, offset odd_arr
```

```
second:
mov eax, [esi]
test eax, 1
jz even_num
```

```
mov [ebx], eax
add ebx, 4
inc oddCount
jmp inc_num
```

```

even_num:
mov [edi], eax
add edi, 4
inc evenCount

inc_num:
add esi, 4

loop second

mov eax, evenCount
call writedec
mov edx, offset str4
call writestring
mov edx, offset str2
call writestring

mov edx, offset str4

mov esi, offset even_arr
mov ecx, evenCount

print_even:
mov eax, [esi]
call writeint
call writestring
add esi, 4
loop print_even

call crlf

mov eax, oddCount
call writedec
mov edx, offset str4
call writestring
mov edx, offset str3
call writestring

mov edx, offset str4

mov esi, offset odd_arr
mov ecx, oddCount

print_odd:
mov eax, [esi]
call writeint
call writestring
add esi, 4
loop print_odd

call crlf

exit

```

```
main ENDP
END main
```

Screenshot:

```
Enter a number: 24
Enter a number: -26
Enter a number: 1109
Enter a number: 53
Enter a number: -5
2 even numbers: +24 -26
3 odd numbers: +1109 +53 -5
```

2. Write an assembly program that reads two unsigned integers from the user and performs the following bitwise operations:

AND: Display the result of the bitwise AND operation.

OR: Display the result of the bitwise OR operation.

XOR: Display the result of the bitwise XOR operation.

NOT: Display the result of the bitwise NOT operation for the first integer.

Print the output in binary.

```
Microsoft Visual Studio Debug Console
Enter the first unsigned integer: 24042404
Enter the second unsigned integer: 11091109
First number: 0000 0001 0110 1110 1101 1011 1010 0100
Second number: 0000 0000 1010 1001 0011 1100 1010 0101
AND result: 0000 0000 0010 1000 0001 1000 1010 0100
OR result: 0000 0001 1110 1111 1111 1111 1010 0101
XOR result: 0000 0001 1100 0111 1110 0111 0000 0001
NOT result of the first integer: 1111 1110 1001 0001 0010 0100 0101 1011

C:\Users\dsankalpa\Desktop\COE251_Template\COE251_Template\Debug\Project.exe (p
Press any key to close this window . . .
```

Solution:

TITLE Name/Purpose Here

```
INCLUDE Irvine32.inc
```

```
.data
```

```
str1 byte "Enter the first unsigned integer: ", 0
str2 byte "Enter the second unsigned integer: ", 0
str3 byte "First number: ", 0
str4 byte "Second number: ", 0
str5 byte "AND result: ", 0
str6 byte "OR result: ", 0
str7 byte "XOR result: ", 0
str8 byte "NOT result of the first integer: ", 0
```

```
.code
```

```
main PROC
```

```
    mov edx, offset str1
    call writestring
    call readint
    mov ebx, eax
```

```
mov edx, offset str2
call writestring
call readint
mov ecx, eax
```

```
mov edx, offset str3
call writestring
mov eax, ebx
call writebin
call crlf
```

```
mov edx, offset str4
call writestring
mov eax, ecx
call writebin
call crlf
```

```
mov edx, offset str5
call writestring
AND eax, ebx
call writebin
call crlf
```

```
mov edx, offset str6
call writestring
mov eax, ecx
OR eax, ebx
call writebin
call crlf
```

```
mov edx, offset str7
call writestring
mov eax, ecx
XOR eax, ebx
call writebin
call crlf
```

```
mov edx, offset str8
call writestring
mov eax, ebx
NOT eax
call writebin
call crlf
```

```
exit
main ENDP
END main
```

Screenshot:

```

Enter the first unsigned integer: 24042404
Enter the second unsigned integer: 11091109
First number: 0000 0001 0110 1110 1101 1011 1010 0100
Second number: 0000 0000 1010 1001 0011 1100 1010 0101
AND result: 0000 0000 0010 1000 0001 1000 1010 0100
OR result: 0000 0001 1110 1111 1111 1111 1010 0101
XOR result: 0000 0001 1100 0111 1110 0111 0000 0001
NOT result of the first integer: 1111 1110 1001 0001 0010 0100 0101 1011

```

3. Write an assembly program that checks whether a positive unsigned number entered by the user is prime or not. Assume the user always inputs a positive integer.

```

Microsoft Visual Studio Debug Console
Enter a positive unsigned integer: 27
27 is not prime.

C:\Users\dsankalpa\Desktop\COE251_Template\COE251_Te
Press any key to close this window . . .

```

```

Microsoft Visual Studio Debug Console
Enter a positive unsigned integer: 1109
1109 is prime.

C:\Users\dsankalpa\Desktop\COE251_Template\COE251
Press any key to close this window . . .

```

Solution:

TITLE Name/Purpose Here

INCLUDE Irvine32.inc

.data

```

    str1 byte "Enter a positive unsigned integer: ", 0
    str2 byte " is not prime", 0
    str3 byte " is prime", 0
    store dword ?

```

.code

main PROC

```

    mov edx, offset str1
    call writestring
    call readint
    mov store, eax

```

```

    cmp store, 2
    jl not_prime

```

```

    mov ecx, 2

```

```

    check_prime:
    cmp ecx, store

```

```

    jge is_prime

    mov eax, store
    xor edx, edx
    div ecx

    cmp edx, 0
    je not_prime

    inc ecx
    jmp check_prime

not_prime:
    mov eax, store
    call writedec
    mov edx, offset str2
    call writestring
    jmp end_program

is_prime:
    mov eax, store
    call writedec
    mov edx, offset str3
    call writestring

end_program:
    call crlf

    exit
main ENDP
END main

```

My Code: -

```

TITLE Name/Purpose Here

INCLUDE Irvine32.inc

.data
    str1 byte "Enter a positive unsigned integer: ", 0
    str2 byte " is a prime.", 0
    str3 byte " is not a prime.", 0
    save dword ?

.code
main PROC
    mov edx, offset str1
    call writestring
    call readint
    mov save, eax

    cmp save, 2
    jl notPrime

    mov ecx, 2

checkPrime:
    cmp ecx, save

```

```

    jge isPrime

    mov edx, 0
    mov eax, save
    div ecx

    cmp edx, 0
    je notPrime

    inc ecx
    jmp checkPrime

notPrime:
    mov eax, save
    call writedec
    mov edx, offset str3
    call writestring
    jmp endProgram

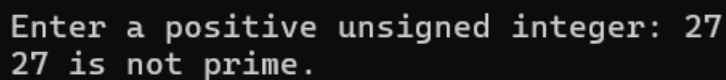
isPrime:
    mov eax, save
    call writedec
    mov edx, offset str2
    call writestring
    jmp endProgram

endProgram:
    call crlf

    exit
main ENDP
END main

```

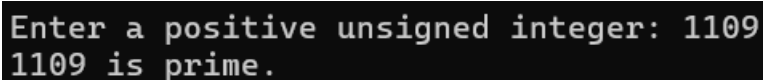
Screenshot:



```

Enter a positive unsigned integer: 27
27 is not prime.

```

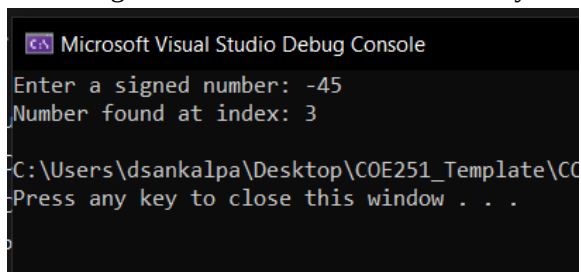


```

Enter a positive unsigned integer: 1109
1109 is prime.

```

4. Given the following 5-element signed array stored in memory: **myArray DWORD 10, -25, 30, -45, 60**. Write an assembly program that asks the user to enter a signed number. The program should check if the number is found in the array and, if so, print the index of the number. If the number is not found, display a message indicating that the number is not in the array.



```

Microsoft Visual Studio Debug Console
Enter a signed number: -45
Number found at index: 3
C:\Users\dsankalpa\Desktop\COE251_Template\CO
Press any key to close this window . . .

```

```
Microsoft Visual Studio Debug Console
Enter a signed number: 100
Number not found in the array.

C:\Users\dsankalpa\Desktop\COE251_Templa
Press any key to close this window . . .
```

Solution:

TITLE Name/Purpose Here

INCLUDE Irvine32.inc

.data

```
str1 byte "Enter a signed number: ", 0
str2 byte "Number found at index: ", 0
str3 byte "Number not found in the array.", 0
myArray dword 10,-25,30,-45,60
store dword ?
```

.code

main PROC

```
mov edx, offset str1
call writestring
call readint
mov store, eax
```

```
mov ebx, 0
mov esi, offset myArray
mov ecx, lengthof myArray
```

```
again:
mov eax, [esi]
```

```
cmp eax, store
je found
```

```
inc ebx
add esi, 4
loop again
```

```
not_found:
mov edx, offset str3
call writestring
jmp end_program
```

```
found:
mov edx, offset str2
call writestring
mov eax, ebx
call writedec
```

```
end_program:
call crlf
```

```
exit
main ENDP
END main
```

My Code: -

```
TITLE Name/Purpose Here

INCLUDE Irvine32.inc

.data
    str1 byte "Enter a signed number: ", 0
    str2 byte "Number found at index: ", 0
    str3 byte "Number not found in the array.", 0
    myArray dword 10,-25,30,-45,60
    i dword 0

.code
main PROC
    mov edx, offset str1
    call writestring
    call readint

    mov esi, offset myArray
    mov ecx, lengthof myArray

again:
    mov ebx, [esi]
    inc i
    cmp eax, ebx
    je found
    add esi, 4
    loop again

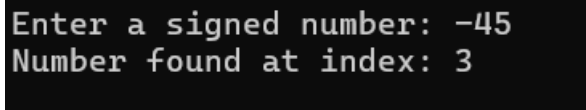
    mov edx, offset str3
    call writestring
    jmp endProgram

found:
    mov edx, offset str2
    call writestring
    mov eax, i
    call writedec
    jmp endProgram

endProgram:
    call crlf

    exit
main ENDP
END main
```

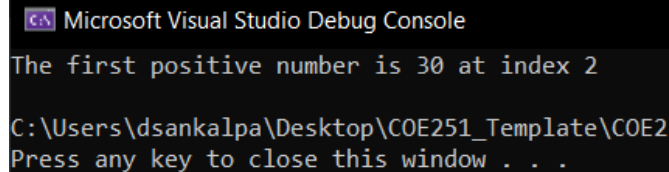
Screenshot:



```
Enter a signed number: -45
Number found at index: 3
```

```
Enter a signed number: 100
Number not found in the array.
```

5. Given the following 6-element signed array stored in memory: **myArray DWORD -10, -25, 30, -45, 60, -5**
Write an assembly program that iterates through the array and finds **the first positive number**. Once the first positive number is found, display its value and index. If no positive number is found, display a message indicating that there are no positive numbers in the array.



```
Microsoft Visual Studio Debug Console
The first positive number is 30 at index 2
C:\Users\dsankalpa\Desktop\COE251_Template\COE2
Press any key to close this window . . .
```

Solution:

TITLE Name/Purpose Here

INCLUDE Irvine32.inc

.data

```
str1 byte "The first positive number is ", 0
str2 byte " at index ", 0
str3 byte "There is no positive numbers in the array.", 0
myArray dword -10,-25,-30,-45,-60,-5
```

.code

main PROC

```
xor ebx, ebx
mov esi, offset myArray
mov ecx, lengthof myArray
```

again:

```
mov eax, [esi]
add esi, 4
```

```
cmp eax, 0
jle not_positive
```

```
mov edx, offset str1
call writestring
call writedec
```

```
mov edx, offset str2
call writestring
mov eax, ebx
call writedec
call crlf
jmp end_program
```

```
not_positive:
inc ebx
```

loop again

mov edx, offset str3
call writestring
call crlf

end_program:

exit
main ENDP
END main

My Code: -

```
TITLE Name/Purpose Here

INCLUDE Irvine32.inc

.data
    str1 byte "The first positive number is ", 0
    str2 byte " at index ", 0
    str3 byte "No positive numbers were found to be in the array.", 0
    myArray dword -10,-25,30,-45,60,-5
    i dword 0

.code
main PROC
    mov esi, offset myArray
    mov ecx, lengthof myArray

again:
    mov eax, [esi]
    cmp eax, 0
    jg positiveNum
    add esi, 4
    inc i
    loop again

    mov edx, offset str3
    call writestring
    jmp endProgram

positiveNum:
    mov edx, offset str1
    call writestring
    call writedec
    mov edx, offset str2
    call writestring
    mov eax, i
    call writedec
    jmp endProgram

endProgram:
    call crlf

    exit
main ENDP
END main
```

Screenshot:

```
The first positive number is 30 at index 2
```